

MARAN PMS — Prompt 47: Grupos — Folio Grupal y Cuenta Madre

Contexto arquitectónico

Este prompt implementa una capa nueva sobre el módulo de grupos existente. NO modifica la lógica actual de reservas individuales — la extiende. Ejecutar después del Prompt 46.

Archivos principales

- `shared/schema.ts` — nuevas tablas
 - `server/storage.ts` — nuevas funciones
 - `server/routes.ts` — nuevos endpoints
 - `client/src/pages/group-detail.tsx` — nuevo tab “Folio Grupal”
-

SECCIÓN 1 — NUEVAS TABLAS EN DB

1.1 Tabla `group_charges` — cargos del folio grupal

```
CREATE TABLE group_charges (  
  id VARCHAR PRIMARY KEY DEFAULT gen_random_uuid(),  
  group_id VARCHAR NOT NULL REFERENCES groups(id),  
  description TEXT NOT NULL,  
  amount DECIMAL(10,2) NOT NULL,  
  date DATE NOT NULL,  
  category TEXT NOT NULL DEFAULT 'otros',  
  billing_target TEXT NOT NULL DEFAULT 'group', -- 'group' | 'guest' | 'company'  
  reservation_id VARCHAR, -- si billing_target = 'guest', qué reserva  
  created_by VARCHAR,  
  created_at TIMESTAMP DEFAULT NOW()  
);
```

1.2 Tabla `group_payments` — pagos del folio grupal

```
CREATE TABLE group_payments (
  id VARCHAR PRIMARY KEY DEFAULT gen_random_uuid(),
  group_id VARCHAR NOT NULL REFERENCES groups(id),
  amount DECIMAL(10,2) NOT NULL,
  method TEXT NOT NULL,
  date DATE NOT NULL,
  reference TEXT,
  distribution TEXT NOT NULL DEFAULT 'equal',
  -- 'equal' | 'proportional_nights' | 'proportional_rate' | 'manual'
  distribution_detail JSONB, -- para distribución manual: { reservationId: amoun
  received_by VARCHAR,
  notes TEXT,
  created_at TIMESTAMP DEFAULT NOW()
);
```

En `shared/schema.ts` agregar ambas tablas con sus tipos Drizzle e insert schemas.

SECCIÓN 2 — BACKEND

2.1 Nuevas funciones en `storage.ts`

```
// Cargos grupales
async createGroupCharge(charge: InsertGroupCharge): Promise<GroupCharge>
async getGroupCharges(groupId: string): Promise<GroupCharge[]>
async deleteGroupCharge(id: string): Promise<boolean>

// Pagos grupales (nuevo modelo)
async createGroupPayment(payment: InsertGroupPayment): Promise<GroupPayment>
async getGroupPayments(groupId: string): Promise<GroupPayment[]>

// Transferir cargo de reserva individual al folio grupal
async transferChargeToGroup(chargeId: string, groupId: string): Promise<GroupChar
// Implementación: copiar el cargo a group_charges con billing_target='group'
// y marcarlo como transferido en charges (agregar campo transferred_to_group: bo

// Folio consolidado del grupo
async getGroupFolio(groupId: string): Promise<GroupFolioData>
// Retorna: group_charges + totales de reservas individuales + group_payments + b
```

2.2 Nuevos endpoints en `routes.ts`

```
// Cargos grupales
```

```

GET    /api/groups/:id/folio          → getGroupFolio (folio consolidado)
POST   /api/groups/:id/charges       → createGroupCharge
DELETE /api/groups/:id/charges/:chargeId → deleteGroupCharge

// Pagos grupales (reemplaza el endpoint actual, mantiene compatibilidad)
POST   /api/groups/:id/payment       → createGroupPayment (nuevo modelo)
GET     /api/groups/:id/payments     → getGroupPayments

// Transferencia de cargos
POST   /api/groups/:id/transfer-charge → transferChargeToGroup
// body: { chargeId, reservationId }

```

2.3 Lógica de distribución de pagos

Al crear un pago grupal (`POST /api/groups/:id/payment`), la distribución se calcula y guarda en `distribution_detail`:

```

async distributeGroupPayment(
  groupId: string,
  totalAmount: number,
  distribution: "equal" | "proportional_nights" | "proportional_rate" | "manual",
  manualDetail?: Record<string, number> // reservationId → monto
): Promise<Record<string, number>> {

  const group = await this.getGroup(groupId);
  const activeRes = group.reservations.filter(
    r => r.status === "confirmed" || r.status === "checked_in"
  );

  if (distribution === "equal") {
    const perRoom = totalAmount / activeRes.length;
    return Object.fromEntries(activeRes.map(r => [r.id, perRoom]));
  }

  if (distribution === "proportional_nights") {
    const totalNights = activeRes.reduce((s, r) => s + (r.nights || 1), 0);
    return Object.fromEntries(activeRes.map(r => [
      r.id, totalAmount * ((r.nights || 1) / totalNights)
    ]));
  }

  if (distribution === "proportional_rate") {
    const totalCost = activeRes.reduce((s, r) => s + parseFloat(r.totalRoomAmount));
    return Object.fromEntries(activeRes.map(r => [
      r.id, totalAmount * (parseFloat(r.totalRoomAmount || "0") / totalCost)
    ]));
  }
}

```

```

    }

    if (distribution === "manual" && manualDetail) {
        return manualDetail;
    }

    // fallback: equal
    const perRoom = totalAmount / activeRes.length;
    return Object.fromEntries(activeRes.map(r => [r.id, perRoom]));
}

```

El `distribution_detail` (JSON) se guarda en `group_payments` para auditoría. Los pagos individuales a cada reserva se siguen creando en la tabla `payments` existente para mantener compatibilidad con los folios individuales.

2.4 Folio consolidado

`getGroupFolio` retorna:

```

type GroupFolioData = {
    group: GroupWithDetails;

    // Cargos directos al grupo (group_charges con billing_target='group')
    groupCharges: GroupCharge[];
    groupChargesTotal: number;

    // Resumen por reserva individual
    reservations: Array<{
        reservationId: string;
        guestName: string;
        roomNumber: string;
        nights: number;
        accommodationTotal: number;
        extrasTotal: number;           // cargos extras de la reserva
        paymentsTotal: number;         // pagos ya aplicados a esta reserva
        balance: number;
    }>;

    // Pagos grupales registrados
    groupPayments: GroupPayment[];
    groupPaymentsTotal: number;

    // Totales consolidados
    totals: {
        accommodation: number;         // suma de todas las reservas
        groupCharges: number;           // cargos del folio grupal
    };
}

```

```

    extras: number;           // cargos extras de reservas individuales
    payments: number;         // todos los pagos (grupales + individuales)
    balance: number;          // lo que falta pagar
  };
};

```

SECCIÓN 3 — FRONTEND: TAB “FOLIO GRUPAL”

3.1 Nuevo tab en GroupDetailPage

En `client/src/pages/group-detail.tsx`, agregar un tercer tab al `<Tabs>` :

```

<TabsTrigger value="folio" data-testid="tab-folio">
  <DollarSign className="mr-2 h-4 w-4" />
  Folio Grupal
</TabsTrigger>

```

3.2 Contenido del tab folio

```

<TabsContent value="folio">
  <div className="space-y-4">

    {/* Resumen totales */}
    <Card>
      <CardHeader className="pb-3">
        <CardTitle className="text-base">Resumen Financiero del Grupo</CardTitle>
      </CardHeader>
      <CardContent>
        <div className="grid grid-cols-2 md:grid-cols-4 gap-4 text-sm">
          <div>
            <p className="text-muted-foreground">Alojamiento</p>
            <p className="font-semibold text-lg">${folio?.totals.accommodation.to
          </div>
          <div>
            <p className="text-muted-foreground">Cargos extras</p>
            <p className="font-semibold text-lg">${folio?.totals.extras.toLocales
          </div>
          <div>
            <p className="text-muted-foreground">Pagos recibidos</p>
            <p className="font-semibold text-lg text-green-600">${folio?.totals.p
          </div>
        </div>
      </CardContent>
    </Card>
  </div>
</TabsContent>

```

```

        <p className="text-muted-foreground">Saldo</p>
        <p className={`font-semibold text-lg ${folio?.totals.balance > 0 ? "t
            ${folio?.totals.balance.toLocaleString()}
        </p>
    </div>
</div>
</CardContent>
</Card>

{/* Cargos del folio grupal */}
<Card>
    <CardHeader className="flex flex-row items-center justify-between pb-3">
        <CardTitle className="text-base">Cargos del Grupo</CardTitle>
        <Button size="sm" onClick={() => setShowAddGroupChargeDialog(true)}>
            <Plus className="h-4 w-4 mr-1" /> Agregar cargo
        </Button>
    </CardHeader>
    <CardContent>
        {/* tabla de group_charges */}
    </CardContent>
</Card>

{/* Desglose por reserva */}
<Card>
    <CardHeader className="pb-3">
        <CardTitle className="text-base">Desglose por Habitación</CardTitle>
    </CardHeader>
    <CardContent>
        {/* tabla con cada reserva: hab, huésped, alojamiento, extras, pagos, bal
        {/* botón "Transferir cargo al folio" en cada fila */}
    </CardContent>
</Card>

{/* Pagos grupales */}
<Card>
    <CardHeader className="flex flex-row items-center justify-between pb-3">
        <CardTitle className="text-base">Pagos Registrados</CardTitle>
        <Button size="sm" onClick={() => setShowGroupPaymentDialog(true)}>
            <CreditCard className="h-4 w-4 mr-1" /> Registrar pago
        </Button>
    </CardHeader>
    <CardContent>
        {/* lista de group_payments con método, monto, fecha, distribución */}
    </CardContent>
</Card>

</div>

```

```
</TabsContent>
```

3.3 Dialog de pago grupal actualizado

Reemplazar el dialog de pago grupal existente con uno que incluya la distribución manual:

```
// Agregar opción manual al Select de distribución:
<SelectItem value="equal">Partes iguales</SelectItem>
<SelectItem value="proportional_nights">Proporcional por noches</SelectItem>
<SelectItem value="proportional_rate">Proporcional por tarifa</SelectItem>
<SelectItem value="manual">Manual (asignar por habitación)</SelectItem>

// Cuando distribution === "manual", mostrar una fila por reserva activa:
{groupPaymentDistribution === "manual" && (
  <div className="space-y-2">
    <Label>Asignación manual</Label>
    {activeReservations.map(res => (
      <div key={res.id} className="flex items-center gap-2">
        <span className="text-sm w-24">Hab. {res.room?.roomNumber}</span>
        <Input
          type="number"
          step="0.01"
          value={manualDistribution[res.id] || ""}
          onChange={(e) => setManualDistribution({
            ...manualDistribution,
            [res.id]: parseFloat(e.target.value) || 0
          })}
          className="w-32"
        />
      </div>
    ))}
    {/* mostrar suma vs total ingresado */}
    <p className="text-xs text-muted-foreground">
      Asignado: ${Object.values(manualDistribution).reduce((a, b) => a + b, 0)}.to
      / Total: ${groupPaymentAmount}
    </p>
  </div>
)}
}
```

3.4 Dialog de transferencia de cargo

Al hacer clic en "Transferir al folio grupal" en una fila de cargo de una reserva individual:

```
// Abre un confirm dialog:
```

```
"¿Mover el cargo '[descripción]' (${monto}) al folio del grupo [nombre]?"
// Botones: Cancelar / Transferir

// Mutation:
const transferChargeMutation = useMutation({
  mutationFn: ({ chargeId }: { chargeId: string }) =>
    apiRequest("POST", `/api/groups/${groupId}/transfer-charge`, { chargeId }),
  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ["/api/groups", groupId] });
    toast({ title: "Cargo transferido al folio grupal" });
  }
});
```

NOTAS TÉCNICAS

- Las tablas `group_charges` y `group_payments` **son nuevas** — no modifican `charges` ni `payments` existentes.
- El campo `transferred_to_group` en `charges` es opcional para la primera implementación — se puede indicar con una nota en `reference` o `description`.
- Los pagos grupales siguen distribuyéndose a las reservas individuales (tabla `payments`) para mantener compatibilidad con los folios individuales y el cierre de caja.
- `distribution_detail` en `group_payments` es solo para auditoría y mostrar en el historial.
- La factura grupal (`GET /api/groups/:id/invoice`) debe actualizarse para incluir los `group_charges` además de los cargos individuales.
- `billing_target` en `group_charges` puede ser `'group'`, `'guest'` o `'company'`. Para la primera implementación implementar solo `'group'` — los otros quedan como placeholder.
- Usar `requireAuth` en todos los nuevos endpoints.